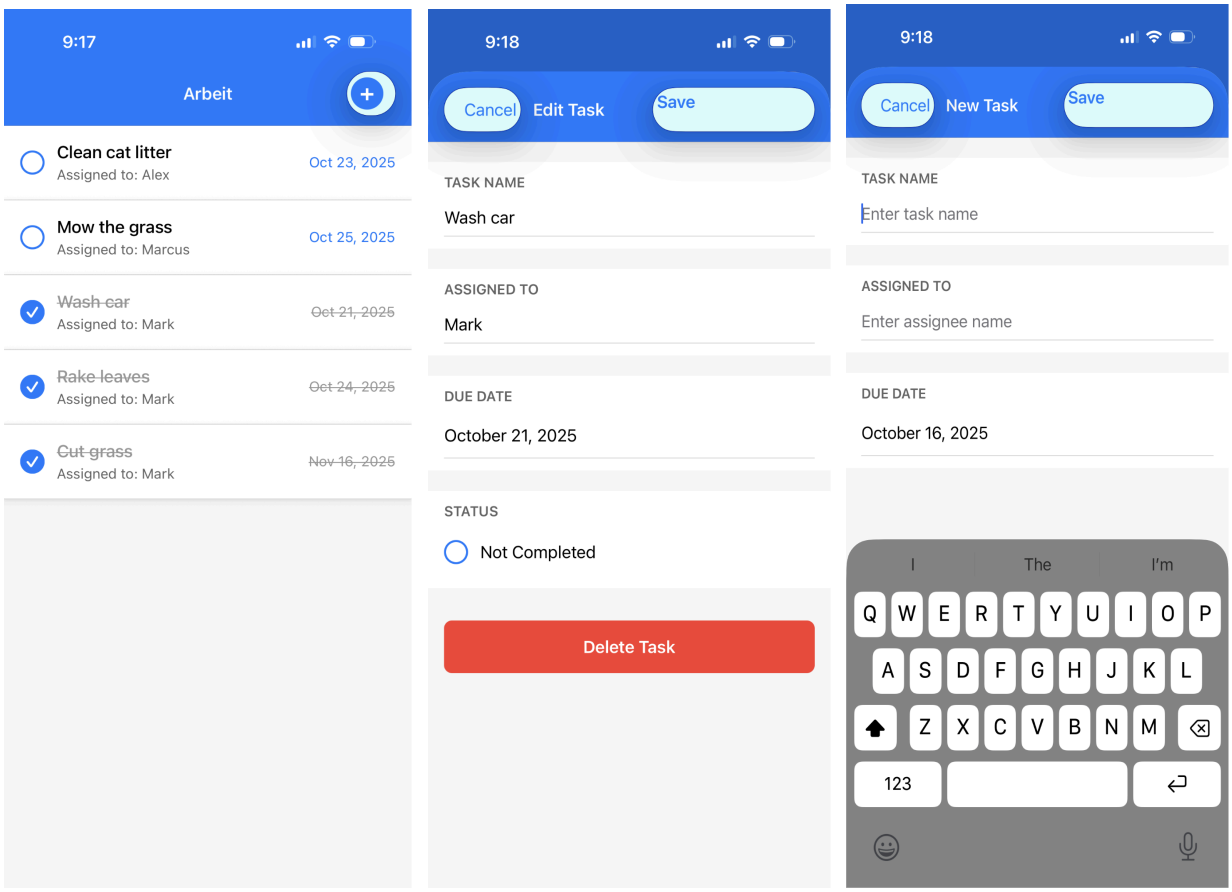
Lab 07: Arbeit Task Manager (React Native)

Last week we learned to save data on the mobile device itself. This week in lab7, you will learn to save data in the cloud using **Firebase Firestore.** You'll learn about app architecture, state management, Firebase integration, and modern React Native development practices. Here is a preview of what the app will look something like:



I realize it's been a little while since we talked about this in class (and we've had fall break), but it's an important topic so it's worth taking the time to read and understand the code. Please don't rush through and just copy/paste code to get this lab done; understanding this better will help a lot in Sprint 4 and beyond.
Note: Please do not forget to add comments to EVERY FUNCTION/METHOD IN YOUR CODE. Even the parts we give you! Good comments help you debug quicker, and also helps us assess your understanding.

Part 0: Initial Setup; Firebase

Initial Setup

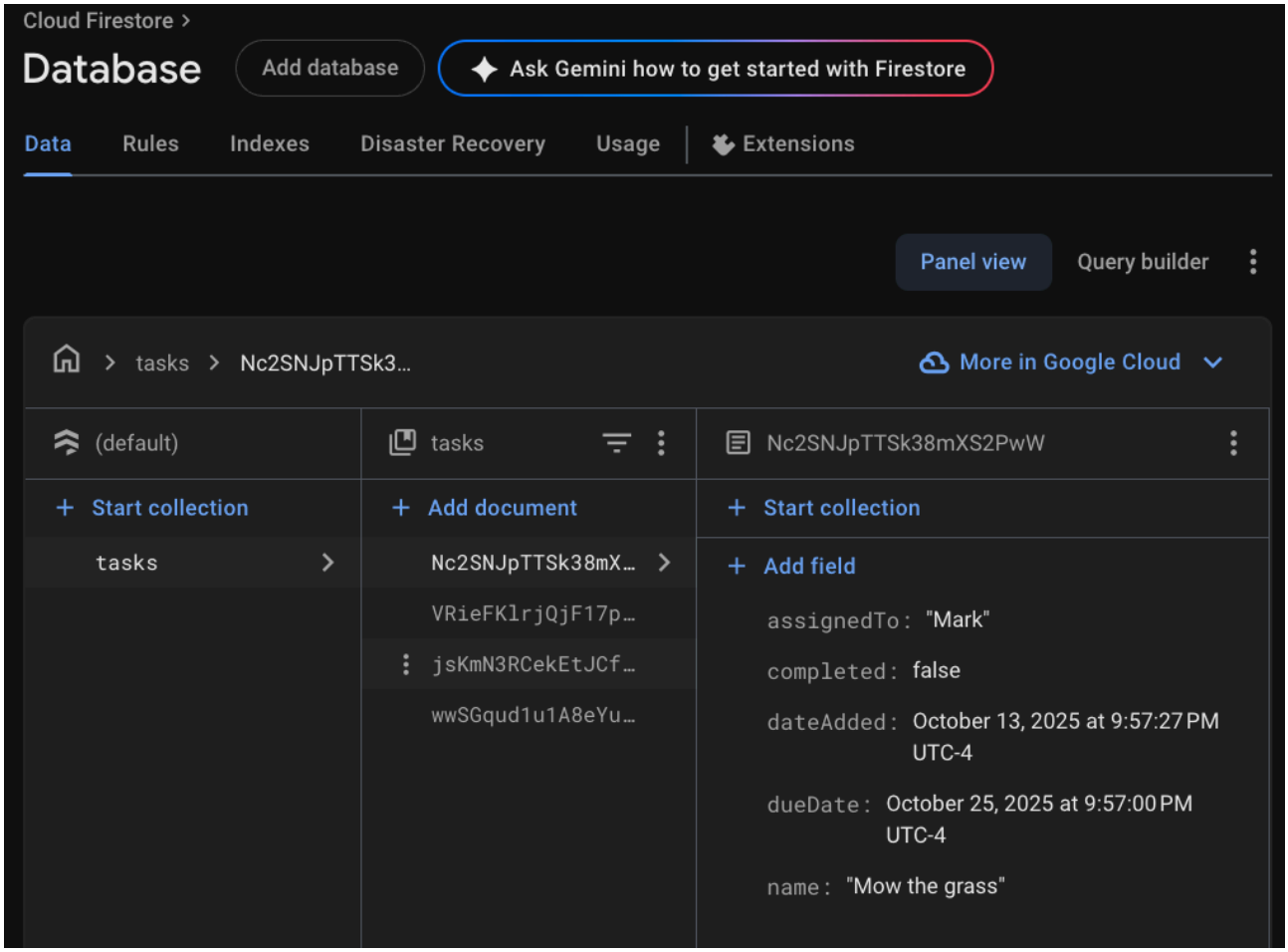
1. Create a new Expo project called `ArbeitRN` and change into it:

```shell
npx create-expo-app@latest ArbeitRN --template blank-typescript
cd ArbeitRN
```

2. Next we need to create a Firebase Firestore:

    - Go to the Firebase Console
    - Create a new project named "Arbeit"
3. Create a Firestore database (start in test mode since we are in development) and populate it with two test records in a collection named "`tasks`". Each document should contain the following fields: `name` (string), `assignedTo` (string), `completed` (boolean), `dueDate` (timestamp), and `dateAdded` (timestamp). ⚠️ **Note:** Make sure the date fields are set to the **timestamp** type in Firestore, not string! The structure should look

like this:



4. Add a **web app** to the project called `ArbeitRN`. (Yes, web app -- the Firebase JS SDK we're about to install works from any JavaScript environment, and Expo apps register as web apps in the Firebase console.)
    - After you register the web app, Firebase will show you an "Add Firebase SDK" step with a snippet of `firebaseConfig` values. Copy that snippet -- we'll use it in a moment.

5. Install Firebase and the two packages we'll need for navigation and date-picking:

```shell
npx expo install firebase @react-navigation/native @react-navigation/native-stack
@react-navigation/bottom-tabs react-native-screens react-native-safe-area-context
@react-native-community/datetimepicker @expo/vector-icons
```

6. Create a `config` folder at the project root and a file `config/firebase.ts` inside it. Paste in the config values you copied from the Firebase console, then export an initialized `db`:

```TypeScript
import { initializeApp } from 'firebase/app';
import { getFirestore } from 'firebase/firestore';

const firebaseConfig = {
  // ...paste the values from the Firebase console here...
};

const app = initializeApp(firebaseConfig);
export const db = getFirestore(app);
```

7. Create three more folders inside your project: `models`, `viewModels`, and `views` -- the same organizational pattern we've used since Lab 02.

Part 1: Creating the Task Model

Create `models/TaskItem.ts` and add:

```TypeScript
export interface TaskItem {
  id?: string;
  name: string;
  assignedTo: string;
  dueDate: Date;
  dateAdded: Date;
  completed: boolean;
}

export function isValidTask(name: string, assignedTo: string): boolean {
  return name.trim().length > 0 && assignedTo.trim().length > 0;
}

export function compareTasksByName(a: TaskItem, b: TaskItem): number {
  return a.name.localeCompare(b.name);
}

export function compareTasksByDueDate(a: TaskItem, b: TaskItem): number {
  return a.dueDate.getTime() - b.dueDate.getTime();
}

export function areTasksEqual(a: TaskItem, b: TaskItem): boolean {
```

```
  return (
    a.name === b.name &&
    a.assignedTo === b.assignedTo &&
    a.dueDate.getTime() === b.dueDate.getTime()
  );
}
```

Understanding the code

⚠️ **Naming note:** the older version of this lab named this model `Task`. That's fine in TypeScript on its own, but it's also the name of a very common concept in async code (`Promise`-shaped work items), and the Swift version had to rename it to `TaskItem` to avoid a hard collision with Swift's concurrency `Task`. We're going with `TaskItem` in the RN version too, for consistency across the two versions of the class and because it reads more clearly.

- **`id?: string`**: optional because new tasks don't have an ID until Firestore creates one at insert time. This is the RN equivalent of Swift's `@DocumentID var id: String?`.
- **Pure comparator functions**: instead of Swift's `Comparable` protocol conformance, we export plain functions that fit the `(a, b) => number` shape `Array.prototype.sort()` wants. Same idea, different syntax.
- **`isValidTask`**: separated from the interface itself so we can call it from both the New Task and Edit Task screens without duplicating validation logic.

Notice this whole file has zero React and zero Firebase imports -- that's deliberate. It's pure logic that will be trivial to unit-test at the end of the lab.

Part 2: Creating the Repository + Provider

In the older version of this lab, there was a `TaskRepository` class doing the Firestore CRUD, plus two separate hooks (`useTask` and `useTasks`) glued together with a shared singleton repository. That was a lot of moving parts. We're going to collapse the repository, the singleton, and both hooks into a single `TasksProvider` that uses the same Context + Provider + custom-hook pattern we used in Labs 03 and 06.

This also matches the direction the Swift F26 version took -- it consolidated two view models into one because `@Observable` no longer needs the elaborate Combine pipeline the older code was built around.

Create `viewModels/TasksContext.tsx`:

None

```
import React, { createContext, useContext, useEffect, useState, ReactNode } from
'react';
import {
  collection,
  onSnapshot,
  addDoc,
  updateDoc,
  deleteDoc,
  doc,
  Timestamp,
} from 'firebase/firestore';
import { db } from '../config/firebase';
import { TaskItem } from '../models/TaskItem';

const COLLECTION = 'tasks';

type TasksContextValue = {
  tasks: TaskItem[];
  loading: boolean;
  addTask: (task: Omit<TaskItem, 'id'>) => Promise<void>;
  updateTask: (task: TaskItem) => Promise<void>;
  deleteTask: (id: string) => Promise<void>;
  toggleCompleted: (task: TaskItem) => Promise<void>;
};

const TasksContext = createContext<TasksContextValue | undefined>(undefined);

export function TasksProvider({ children }: { children: ReactNode }) {
  const [tasks, setTasks] = useState<TaskItem[]>([]);
  const [loading, setLoading] = useState(true);

  // Subscribe to real-time updates from Firestore. The unsubscribe returned
  // by onSnapshot is what we call on unmount to tear the listener down --
  // the RN equivalent of the Swift version removing its ListenerRegistration
  // in deinit.
  useEffect(() => {
    const unsubscribe = onSnapshot(
      collection(db, COLLECTION),
      (snapshot) => {
        const next: TaskItem[] = snapshot.docs.map((d) => {
          const data = d.data();
          return {
            id: d.id,
            name: data.name,
            assignedTo: data.assignedTo,
            dueDate: data.dueDate?.toDate() ?? new Date(),
            dateAdded: data.dateAdded?.toDate() ?? new Date(),
            completed: data.completed ?? false,
```

```
        };
      });
      // Incomplete first, then by due date ascending.
      next.sort((a, b) => {
        if (a.completed !== b.completed) return a.completed ? 1 : -1;
        return a.dueDate.getTime() - b.dueDate.getTime();
      });
      setTasks(next);
      setLoading(false);
    },
    (error) => {
      console.error('Firestore listener error:', error);
      setLoading(false);
    }
  );
  return unsubscribe;
}, []);

const addTask = async (task: Omit<TaskItem, 'id'>) => {
  await addDoc(collection(db, COLLECTION), {
    ...task,
    dueDate: Timestamp.fromDate(task.dueDate),
    dateAdded: Timestamp.fromDate(task.dateAdded),
  });
};

const updateTask = async (task: TaskItem) => {
  if (!task.id) throw new Error('Cannot update a task without an id');
  await updateDoc(doc(db, COLLECTION, task.id), {
    name: task.name,
    assignedTo: task.assignedTo,
    dueDate: Timestamp.fromDate(task.dueDate),
    completed: task.completed,
  });
};

const deleteTask = async (id: string) => {
  await deleteDoc(doc(db, COLLECTION, id));
};

const toggleCompleted = async (task: TaskItem) => {
  if (!task.id) throw new Error('Cannot toggle a task without an id');
  await updateDoc(doc(db, COLLECTION, task.id), { completed: !task.completed });
};

return (
  <TasksContext.Provider value={{ tasks, loading, addTask, updateTask,
deleteTask, toggleCompleted }}>
```

```
      {children}
    </TasksContext.Provider>
  );
}

export function useTasks(): TasksContextValue {
  const ctx = useContext(TasksContext);
  if (!ctx) throw new Error('useTasks must be used inside a TasksProvider');
  return ctx;
}
```

Understanding the code

- ⚠️ **Note on Firestore Rules:** Firestore security rules created in test mode expire automatically after 30 days. If your app suddenly stops loading tasks or throws permission errors later, check your database rules in the Firebase console!
- **One listener for the whole app**: because the Firestore subscription lives in `TasksProvider`, we set it up exactly once at app launch. Compare this to the older version where each place that called `useTasks()` set up its own listener -- functional, but wasteful.
- **`onSnapshot` returns its unsubscriber**: returning it directly from `useEffect` is the idiomatic React pattern for "clean me up when I unmount." This is what the Swift version does explicitly in `deinit`.
- **Timestamps in, `Date`s out**: Firestore's native date type is `Timestamp`. We convert at the boundary of the repository (`Timestamp.fromDate()` on writes, `data.dueDate?.toDate()` on reads) so the rest of the app can work with plain `Date` objects.
- **`Omit<TaskItem, 'id'>` for `addTask`**: TypeScript's way of saying "the same shape as `TaskItem`, minus the `id` field" -- since Firestore is about to assign one for us.

Part 3: Shared Form Helpers

Both `NewTaskView` and `TaskDetailsView` need to format dates, handle the date-picker's `onChange` event, and show validation/confirmation alerts. Extract those into `viewModels/taskFormHelpers.ts` up front so we don't duplicate them across both screens:

TypeScript

```
import { Alert, Platform } from 'react-native';
import { isValidTask } from '../models/TaskItem';

export function formatDate(date: Date): string {
```

```
  return date.toLocaleDateString('en-US', {
    month: 'long', day: 'numeric', year: 'numeric',
  });
}

export function makeDateChangeHandler(
  setShowPicker: (show: boolean) => void,
  onPick: (date: Date) => void,
) {
  return (_event: unknown, selected?: Date) => {
    // Android auto-dismisses; iOS stays open until we set it to false ourselves.
    setShowPicker(Platform.OS === 'ios');
    if (selected) onPick(selected);
  };
}

export async function saveWithValidation(
  name: string,
  assignedTo: string,
  save: () => Promise<void>,
  onDone: () => void,
) {
  if (!isValidTask(name, assignedTo)) {
    Alert.alert('Validation Error', 'Please fill in both name and assignee.');
    return;
  }
  try {
    await save();
    onDone();
  } catch (error) {
    console.error(error);
    Alert.alert('Error', 'Failed to save task.');
  }
}

export function confirmDelete(
  performDelete: () => Promise<void>,
  onDone: () => void,
) {
  Alert.alert('Delete Task', 'Are you sure you want to delete this task?', [
    { text: 'Cancel', style: 'cancel' },
    {
      text: 'Delete',
      style: 'destructive',
      onPress: async () => {
        try {
          await performDelete();
          onDone();
```

```
        } catch (error) {
          console.error(error);
          Alert.alert('Error', 'Failed to delete task.');
        }
      },
    },
  ]);
}
```

(An earlier version of this lab presented the un-refactored versions of these handlers duplicated inside both screens, then stopped and asked you to spot the duplication and pull them out. That's a valuable exercise -- and if you want the practice, don't peek at this file, write the screens first with the handlers inlined, and only extract afterwards. The finished product is the same either way.)

Part 4: The Views

Create `types.ts` at the project root for the navigation param list:

TypeScript

```
import type { TaskItem } from './models/TaskItem';

export type TasksStackParamList = {
  TasksList: undefined;
  TaskDetails: { taskId: string };
};

export type RootTabParamList = {
  Tasks: undefined;
  NewTask: undefined;
};
```

Notice we pass just a `taskId` string to `TaskDetails`, not the whole `TaskItem` -- the details screen always looks up the freshest copy from context, same pattern we used in `SimpleContacts`.

Step 4.1: TaskRowView

Create `views/TaskRowView.tsx`:

None

```
import React from 'react';
import { View, Text, Pressable, StyleSheet } from 'react-native';
import { Ionicons } from '@expo/vector-icons';
import { TaskItem } from '../models/TaskItem';
import { formatDate } from '../viewModels/taskFormHelpers';

type Props = {
  task: TaskItem;
  onPress: () => void;
  onToggleCompleted: () => void;
};

export default function TaskRowView({ task, onPress, onToggleCompleted }: Props) {
  return (
    <Pressable style={styles.container} onPress={onPress}>
      <Pressable onPress={onToggleCompleted} hitSlop={8}
style={styles.checkboxTap}>
        <Ionicons
          name={task.completed ? 'checkmark-circle' : 'ellipse-outline'}
          size={26}
          color={task.completed ? '#34C759' : '#007AFF'}
        />
      </Pressable>
      <View style={styles.text}>
        <Text style={[styles.name, task.completed &&
styles.strikethrough]}>{task.name}</Text>
        <Text style={styles.assignee}>Assigned to: {task.assignedTo}</Text>
      </View>
      <Text style={[styles.dueDate, task.completed && styles.strikethrough]}>
        {formatDate(task.dueDate)}
      </Text>
    </Pressable>
  );
}

const styles = StyleSheet.create({
  container: {
    flexDirection: 'row', alignItems: 'center', padding: 16,
    backgroundColor: '#fff', borderBottomWidth: StyleSheet.hairlineWidth,
borderBottomColor: '#ccc',
  },
  checkboxTap: { marginRight: 12 },
  text: { flex: 1 },
  name: { fontSize: 17, fontWeight: '500' },
  assignee: { fontSize: 14, color: '#666', marginTop: 2 },
  dueDate: { fontSize: 14, color: '#007AFF', marginLeft: 8 },
  strikethrough: { textDecorationLine: 'line-through', color: '#999' },
});
```

Step 4.2: TasksView

Create `views/TasksView.tsx`:

None

```
import React from 'react';
import { View, Text, FlatList, ActivityIndicator, StyleSheet } from
'react-native';
import { useNavigation } from '@react-navigation/native';
import type { NativeStackNavigationProp } from '@react-navigation/native-stack';
import { useTasks } from '../viewModels/TasksContext';
import TaskRowView from './TaskRowView';
import type { TasksStackParamList } from '../types';

type NavProp = NativeStackNavigationProp<TasksStackParamList, 'TasksList'>;

export default function TasksView() {
  const { tasks, loading, toggleCompleted } = useTasks();
  const navigation = useNavigation<NavProp>();

  if (loading) {
    return (
      <View style={styles.centered}>
        <ActivityIndicator size="large" />
      </View>
    );
  }

  if (tasks.length === 0) {
    return (
      <View style={styles.centered}>
        <Text style={styles.emptyTitle}>No tasks yet</Text>
        <Text style={styles.emptySubtitle}>Head to the New Task tab to create
one</Text>
      </View>
    );
  }

  return (
    <FlatList
      data={tasks}
      keyExtractor={(item) => item.id!}
      renderItem={({ item }) => (
        <TaskRowView
          task={item}
          onPress={() => navigation.navigate('TaskDetails', { taskId: item.id! })}
          onToggleCompleted={() => toggleCompleted(item)}
        />
```

```
      )}
    />
  );
}

const styles = StyleSheet.create({
  centered: { flex: 1, justifyContent: 'center', alignItems: 'center', padding: 24
},
  emptyTitle: { fontSize: 20, fontWeight: '600', color: '#666', marginBottom: 8 },
  emptySubtitle: { fontSize: 16, color: '#999', textAlign: 'center' },
});
```

`useTasks()` here plays the role `@Environment(TasksViewModel.self)` played in the Swift version -- one line to reach into the shared store and pull out whatever the screen needs.

Step 4.3: NewTaskView

Create `views/NewTaskView.tsx`:

```
None
import React, { useState } from 'react';
import { View, Text, TextInput, Pressable, ScrollView, Platform, StyleSheet } from
'react-native';
import DateTimePicker from '@react-native-community/datetimepicker';
import { useNavigation } from '@react-navigation/native';
import type { BottomTabNavigationProp } from '@react-navigation/bottom-tabs';
import { useTasks } from '../viewModels/TasksContext';
import { formatDate, makeDateChangeHandler, saveWithValidation } from
'../viewModels/taskFormHelpers';
import type { RootTabParamList } from '../types';

type NavProp = BottomTabNavigationProp<RootTabParamList, 'NewTask'>;

export default function NewTaskView() {
  const { addTask } = useTasks();
  const navigation = useNavigation<NavProp>();

  const [name, setName] = useState('');
  const [assignedTo, setAssignedTo] = useState('');
  const [dueDate, setDueDate] = useState(new Date());
  const [showPicker, setShowPicker] = useState(false);

  const onDateChange = makeDateChangeHandler(setShowPicker, setDueDate);
```

```
  const onSave = () =>
    saveWithValidation(name, assignedTo, async () => {
      await addTask({ name, assignedTo, dueDate, dateAdded: new Date(), completed:
false });
      setName(''); setAssignedTo(''); setDueDate(new Date());
    }, () => navigation.navigate('Tasks'));

  return (
    <ScrollView style={styles.container}>
      <View style={styles.section}>
        <Text style={styles.label}>Task Name</Text>
        <TextInput style={styles.input} placeholder="Enter task name" value={name}
onChangeText={setName} />
      </View>

      <View style={styles.section}>
        <Text style={styles.label}>Assigned To</Text>
        <TextInput style={styles.input} placeholder="Enter assignee name"
value={assignedTo} onChangeText={setAssignedTo} />
      </View>

      <View style={styles.section}>
        <Text style={styles.label}>Due Date</Text>
        <Pressable style={styles.dateButton} onPress={() => setShowPicker((s) =>
!s)}>
          <Text style={styles.dateText}>{formatDate(dueDate)}</Text>
        </Pressable>
        {showPicker && (
          <DateTimePicker
            value={dueDate}
            mode="date"
            display={Platform.OS === 'ios' ? 'spinner' : 'default'}
            onChange={onDateChange}
          />
        )}
      </View>

      <Pressable style={styles.saveButton} onPress={onSave}>
        <Text style={styles.saveButtonText}>Save Task</Text>
      </Pressable>
    </ScrollView>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#f5f5f5' },
  section: { backgroundColor: '#fff', paddingHorizontal: 16, paddingVertical: 12,
marginTop: 20 },
```

```
  label: { fontSize: 13, color: '#666', textTransform: 'uppercase', marginBottom:
8 },
  input: { fontSize: 17, paddingVertical: 8, borderBottomWidth:
StyleSheet.hairlineWidth, borderBottomColor: '#ccc' },
  dateButton: { paddingVertical: 12 },
  dateText: { fontSize: 17, color: '#007AFF' },
  saveButton: { margin: 16, backgroundColor: '#007AFF', padding: 16, borderRadius:
8, alignItems: 'center' },
  saveButtonText: { color: '#fff', fontSize: 17, fontWeight: '600' },
});
```

Notice the whole `handleSave` collapses to one line thanks to `saveWithValidation` -- the same shape validate → save → dismiss on success shows up in Edit too.

Step 4.4: TaskDetailsView

Create `views/TaskDetailsView.tsx`:

None

```
import React, { useState } from 'react';
import { View, Text, TextInput, Pressable, ScrollView, Platform, StyleSheet } from
'react-native';
import DateTimePicker from '@react-native-community/datetimepicker';
import { useNavigation, useRoute } from '@react-navigation/native';
import type { RouteProp } from '@react-navigation/native';
import { useTasks } from '../viewModels/TasksContext';
import type { TasksStackParamList } from '../types';
import {
  formatDate, makeDateChangeHandler, saveWithValidation, confirmDelete,
} from '../viewModels/taskFormHelpers';
import { Ionicons } from '@expo/vector-icons';

type RouteProps = RouteProp<TasksStackParamList, 'TaskDetails'>;

export default function TaskDetailsView() {
  const { tasks, updateTask, deleteTask } = useTasks();
  const navigation = useNavigation();
  const route = useRoute<RouteProps>();
  const task = tasks.find((t) => t.id === route.params.taskId);

  const [name, setName] = useState(task?.name ?? '');
  const [assignedTo, setAssignedTo] = useState(task?.assignedTo ?? '');
  const [dueDate, setDueDate] = useState(task?.dueDate ?? new Date());
  const [completed, setCompleted] = useState(task?.completed ?? false);
```

```
  const [showPicker, setShowPicker] = useState(false);

  if (!task) {
    return (
      <View style={styles.centered}>
        <Text>Task not found.</Text>
      </View>
    );
  }

  const onDateChange = makeDateChangeHandler(setShowPicker, setDueDate);

  const onSave = () =>
    saveWithValidation(
      name, assignedTo,
      () => updateTask({ ...task, name, assignedTo, dueDate, completed }),
      () => navigation.goBack(),
    );

  const onDelete = () =>
    confirmDelete(() => deleteTask(task.id!), () => navigation.goBack());

  return (
    <ScrollView style={styles.container}>
      <View style={styles.section}>
        <Text style={styles.label}>Task Name</Text>
        <TextInput style={styles.input} value={name} onChangeText={setName} />
      </View>

      <View style={styles.section}>
        <Text style={styles.label}>Assigned To</Text>
        <TextInput style={styles.input} value={assignedTo}
onChangeText={setAssignedTo} />
      </View>

      <View style={styles.section}>
        <Text style={styles.label}>Due Date</Text>
        <Pressable onPress={() => setShowPicker((s) => !s)}>
          <Text style={styles.dateText}>{formatDate(dueDate)}</Text>
        </Pressable>
        {showPicker && (
          <DateTimePicker
            value={dueDate}
            mode="date"
            display={Platform.OS === 'ios' ? 'spinner' : 'default'}
            onChange={onDateChange}
          />
        )}
```

```
      </View>

      <View style={styles.section}>
        <Text style={styles.label}>Status</Text>
        <Pressable style={styles.statusRow} onPress={() => setCompleted((c) =>
!c)}>
          <Ionicons
            name={completed ? 'checkmark-circle' : 'ellipse-outline'}
            size={26}
            color={completed ? '#34C759' : '#007AFF'}
          />
          <Text style={styles.statusText}>{completed ? 'Completed' : 'Not
Completed'}</Text>
        </Pressable>
      </View>

      <Pressable style={styles.saveButton} onPress={onSave}>
        <Text style={styles.saveButtonText}>Save Changes</Text>
      </Pressable>
      <Pressable style={styles.deleteButton} onPress={onDelete}>
        <Text style={styles.deleteButtonText}>Delete Task</Text>
      </Pressable>
    </ScrollView>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#f5f5f5' },
  centered: { flex: 1, justifyContent: 'center', alignItems: 'center' },
  section: { backgroundColor: '#fff', paddingHorizontal: 16, paddingVertical: 12,
marginTop: 20 },
  label: { fontSize: 13, color: '#666', textTransform: 'uppercase', marginBottom:
8 },
  input: { fontSize: 17, paddingVertical: 8, borderBottomWidth:
StyleSheet.hairlineWidth, borderBottomColor: '#ccc' },
  dateText: { fontSize: 17, color: '#007AFF', paddingVertical: 12 },
  statusRow: { flexDirection: 'row', alignItems: 'center', gap: 12,
paddingVertical: 8 },
  statusText: { fontSize: 17 },
  saveButton: { margin: 16, backgroundColor: '#007AFF', padding: 16, borderRadius:
8, alignItems: 'center' },
  saveButtonText: { color: '#fff', fontSize: 17, fontWeight: '600' },
  deleteButton: { marginHorizontal: 16, marginBottom: 32, backgroundColor:
'#FF3B30', padding: 16, borderRadius: 8, alignItems: 'center' },
  deleteButtonText: { color: '#fff', fontSize: 17, fontWeight: '600' },
});
```

Because the local state (`name`, `assignedTo`, etc.) is seeded from the fresh `task` looked up in context, edits stay local until the user taps Save -- at which point `updateTask({ ...task, ... })` writes them back through the provider and the Firestore subscription pushes the updated task back into `tasks` on every subscriber's next render.

Part 5: Wiring Up Navigation

Open `App.tsx` and replace the boilerplate with a bottom tab bar (Tasks and New Task) plus a nested native stack for the Tasks tab so we can push into details:

```
None
import React from 'react';
import { NavigationContainer } from '@react-navigation/native';
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import { Ionicons } from '@expo/vector-icons';
import { TasksProvider } from './viewModels/TasksContext';
import TasksView from './views/TasksView';
import NewTaskView from './views/NewTaskView';
import TaskDetailsView from './views/TaskDetailsView';
import type { TasksStackParamList, RootTabParamList } from './types';

const Tab = createBottomTabNavigator<RootTabParamList>();
const TasksStack = createNativeStackNavigator<TasksStackParamList>();

function TasksStackScreen() {
  return (
    <TasksStack.Navigator>
      <TasksStack.Screen name="TasksList" component={TasksView} options={{ title:
'Arbeit' }} />
      <TasksStack.Screen name="TaskDetails" component={TaskDetailsView} options={{
title: 'Task Details' }} />
    </TasksStack.Navigator>
  );
}

export default function App() {
  return (
    <TasksProvider>
      <NavigationContainer>
        <Tab.Navigator
          screenOptions={({ route }) => ({
            headerShown: false,
            tabBarIcon: ({ color, size }) => {
              const icon = route.name === 'Tasks' ? 'checkmark-done-circle' :
'add-circle';
```

```
              return <Ionicons name={icon} size={size} color={color} />;
            },
          })}
        >
          <Tab.Screen name="Tasks" component={TasksStackScreen} />
          <Tab.Screen name="NewTask" component={NewTaskView} options={{ title:
'New Task', headerShown: true }} />
        </Tab.Navigator>
      </NavigationContainer>
    </TasksProvider>
  );
}
```

The `TasksProvider` wraps `NavigationContainer` so every tab and every screen sees the same shared store -- this is the RN equivalent of `AppView`'s `.environment(tasksViewModel)` in the Swift version.

Run the app (`npx expo start`, then press `i` or `a`) and confirm you can create, edit, complete, and delete tasks. Watch the list update in real time -- if you edit a document directly in the Firebase console, it should show up in the app within a second.

Part 6: Testing the Model

The `TaskItem` module has pure logic that's easy to test without hitting Firebase at all. This is why we kept validation and the comparators in `models/TaskItem.ts` rather than baking them into the provider.

Install Jest with the Expo preset (same setup as prior labs):

Shell
```
npx expo install jest-expo jest @types/jest --dev
```

Add a `jest` block and a `test` script to your `package.json`:

JSON
```
"scripts": {
  "test": "jest"
},
"jest": {
  "preset": "jest-expo"
}
```

Also, open your `tsconfig.json` and add `"types": ["jest"]` under `compilerOptions` so TypeScript recognizes Jest's global testing functions (like `describe`, `test`, and `expect`):

JSON

```
"compilerOptions": {
  "strict": true,
  "types": ["jest"]
}
```

Create a `__tests__` folder at the project root and add `taskItem.test.ts`:

TypeScript

```
import {
  isValidTask,
  compareTasksByName,
  compareTasksByDueDate,
  areTasksEqual,
  TaskItem,
} from '../models/TaskItem';

const fixedDate = new Date('2026-01-15T00:00:00Z');

function sample(overrides: Partial<TaskItem> = {}): TaskItem {
  return {
    name: 'Write lab',
    assignedTo: 'Prof H',
    dueDate: fixedDate,
    dateAdded: fixedDate,
    completed: false,
    ...overrides,
  };
}

describe('isValidTask', () => {
  test('rejects an empty name', () => {
    expect(isValidTask('', 'someone')).toBe(false);
  });
  test('rejects an empty assignee', () => {
    expect(isValidTask('something', '')).toBe(false);
  });
  test('rejects whitespace-only fields', () => {
    expect(isValidTask('   ', 'someone')).toBe(false);
    expect(isValidTask('something', '   ')).toBe(false);
  });
  test('accepts both fields present', () => {
```

```
    expect(isValidTask('something', 'someone')).toBe(true);
  });
});

describe('comparators', () => {
  test('compareTasksByName sorts alphabetically', () => {
    const list = [sample({ name: 'Cherry' }), sample({ name: 'Apple' }), sample({
name: 'Banana' })];
    list.sort(compareTasksByName);
    expect(list.map((t) => t.name)).toEqual(['Apple', 'Banana', 'Cherry']);
  });

  test('compareTasksByDueDate sorts earliest-first', () => {
    const a = sample({ name: 'A', dueDate: new Date('2026-03-01') });
    const b = sample({ name: 'B', dueDate: new Date('2026-01-01') });
    const c = sample({ name: 'C', dueDate: new Date('2026-02-01') });
    const list = [a, b, c].sort(compareTasksByDueDate);
    expect(list.map((t) => t.name)).toEqual(['B', 'C', 'A']);
  });
});

describe('areTasksEqual', () => {
  test('ignores completion state and id', () => {
    const a = sample();
    const b = sample({ completed: true });
    expect(areTasksEqual(a, b)).toBe(true);
  });
  test('differs when the name differs', () => {
    expect(areTasksEqual(sample({ name: 'A' }), sample({ name: 'B'
}))).toBe(false);
  });
  test('differs when the due date differs', () => {
    expect(
      areTasksEqual(sample({ dueDate: new Date('2026-01-01') }), sample({ dueDate:
new Date('2026-02-01') }))
    ).toBe(false);
  });
});
```

Run it:

```Shell
npm test
```

All nine tests should pass. ✅

Why the shared `fixedDate`? `areTasksEqual` compares `dueDate` by `getTime()`, and two calls to `new Date()` even nanoseconds apart return different times. If `sample()` used `new Date()` internally, two "identical" samples wouldn't actually be equal and the equality test would fail for a silly reason. Sharing a fixed `Date` keeps the test honest -- same reason the Swift F26 lab uses a fixed `Date(timeIntervalSince1970: 1_700_000_000)` in its test helper.

Why we don't test the provider directly: `TasksProvider` talks to a live Firestore. Unit-testing it requires mocking Firebase's `onSnapshot` / `addDoc` / `updateDoc` primitives and using `@testing-library/react-native`'s `renderHook`. That's a much bigger machinery investment than pulling the pure logic out gets us. If you want to go further, extract a `TaskStore` interface that the provider consumes, and inject a fake implementation for tests. That's a great optional exercise if you're comfortable with dependency injection.

Part 7: Enhancements (Optional Challenges)

The app should be functioning at this point and you'll get full credit, but honestly this is just the start. Some ideas if you have the time:

- **Filter tasks**: add a "show incomplete only" toggle at the top of `TasksView`, backed by a `useState` and a `useMemo` around the filtered list.
- **Sort tasks**: let the user pick between name and due-date order using an `Alert.alert` with two options (same pattern as `SimpleContacts`).
- **Task priority**: add a `priority` field to `TaskItem` (e.g., `'low' | 'medium' | 'high'`), a picker to the two form screens, and color-coding to `TaskRowView`.
- **Search**: add a native search bar to the Tasks tab header using `headerSearchBarOptions` (same pattern as `SimpleContacts`) and filter by name.
- **User authentication**: install `firebase/auth`, add login/signup screens, and scope the `tasks` collection to the current user (`collection(db, 'users', uid, 'tasks')`).

Many, but not all, apps this semester will use Firebase to store some or all of the app's data. In any case, there are great concepts to learn here, so take your time, understand each concept, and don't hesitate to experiment.

Qapla'